Targeted Super-Resolution of Content in Video Clips using Video-Diffusion Models

IE 643 course project · Team Pairadox · Avishi (23B2523)

In real-world scenarios, areas of interest often take up a small portion of the overall image (e.g. a face, sign or number plate). We demonstrate ×4 super-resolution of a user-defined area of interest in a video clip utilising a pre-trained video diffusion prior. The area of interest is tracked, cropped to create a spatial-temporal tube with a constant size and super-resolved using Stable Video Diffusion (SVD). The super-resolved tube is then inserted with a feathered border to produce the final result. This does not involve any weights being trained. The low resolution region of interest provides guidance to the frozen prior via partial noise initialization, using a one-frame look ahead to transition between successive diffusions and maintaining low frequency information to retain colour and structure.

We create low and high resolution tubes of the area of interest using the DAVIS 2017 dataset with ground truth object masks. A data driven approach was taken to choose the regions of interest to evaluate. There is only one hyperparameter to be tuned (strength) which was done on a validation split. We tested on 16 tubes with bicubic and real world degradation. All metrics were calculated on the region of interest only. The video prior significantly outperformed the image prior (Stable Diffusion ×4) in reducing temporal flicker (factor of 4-5 lower tLPIPS) and retained 1.7-2.7dB more PSNR in the region of interest. However, there was only a small perceptual gain over bicubic interpolation. The GAN based prior (Real-ESRGAN), applied on a per-frame basis, still significantly outperformed our method on perceptual metrics. We have analysed why very little detail was added due to the conservative training free conditioning. An interactive tool was created allowing users to highlight an area of interest and receive a comparison with the baseline.

Abstract

If a video is too blurry to read a sign or identify a face, then typically there will be one area that needs to be fixed. For this project, the user will draw a box around the desired target object within the first frame. This object will be four times upscaled using super-resolution across all frames. A pre-trained video diffusion model called Stable Video Diffusion (SVD) is used. This model is completely frozen. The low resolution target was used to guide the video diffusion model through the following methods: denoise starting

with partially noisy input, sequential windows of 14 frames where each window pass on the up scaled frame, and the final output has the rough structure and colours taken from the input.

We created a set of ground truth pairs using data from DAVIS 2017. This gives precise information about where each target object is in every frame using object masks. Test objects were chosen based solely on dataset statistics. The single parameter was optimised using a separate validation split. All metrics were calculated within the target. Across 16 test clips, the video prior had 4-5 times less flickering than the individual frame image prior from our initial experiment. It also remained 1.7-2.7dB closer to the true content. However, the improvement over bicubic interpolation was minimal. A frame-by-frame GAN (Real-ESRGAN) still gives sharper textures. We will explain why this is the case and provide an interactive demo that lets anyone mark their own target video and see how it compares to a baseline.

1. Introduction

Most methods for using video to improve resolution aim to improve the resolution of all pixels in all frames. However, when looking at CCTV or dashcam footage, you may only want to identify one face or number plate. This wastes most of the computational resources by upscaling areas which will not be focused on. The model may also have been trained to give a good overall result but may not give a good result on the specific area that the user is interested in. This project aims to upscale only what the user is interested in to give a good result.

Upscaling small areas of interest is challenging. Classical and regression methods such as BasicVSR++ [8] are trained to reduce the average error in pixel values. If there are multiple high resolution options for a given low resolution input then it will give a result similar to the average of these options, causing blurriness. Diffusion models do not suffer from this issue. They have seen millions of real images so know what real textures look like. This means the generated details are realistic. However, realistic does not equal correct. Generating sharp made-up letters on the number plates can be worse than having a blurry number plate as it is misleading. This issue with balance between realistic and accurate is a theme throughout this project.

Videos require the final output to be consistent over time. Using an image based model to upscale, will produce slightly different details each time and cause a shimmer effect. Video diffusion models (such as Stable Video Diffusion [3]) contain layers which can look across frames and have been trained on real movement so should offer consistent details. Our task was to generate using such a model with the model being a fixed prior. We conditioned the model on the low resolution target and demonstrated how this performed compared to our baseline on the target area.

This report outlines how the project was developed. Firstly an initial experiment was done using an image prior to understand what issues occurred. From this the pipeline was re-created using a video prior and proper evaluation was done. Our main contributions were:

- a pipeline that turns one box on the first frame into a stable, fixed-size crop of the target through the whole clip (a *tube*), restores it, and blends it back into the frame without visible seams;
- a training-free way to steer a frozen Stable Video Diffusion model with the low-resolution tube;
- an evaluation built on real ground truth, with targets chosen from data statistics, a separate validation split for tuning, and metrics measured only inside the target, including two measures of flicker;
- an interactive demo for marking a target in any video and comparing the result with a baseline.

2. Workflow

The project moved through the following stages:

1. **Reading.** Diffusion models and latent diffusion, Stable Video Diffusion, diffusion-based super-resolution for images and video, the standard non-diffusion baselines, and video segmentation with SAM 2.
2. **Experiment 1 (before the prep review).** A first end-to-end version with a frozen *image* prior, a simple tracker and a basic Gradio demo, tried on a single stock video.
3. **Looking critically at Experiment 1.** It had no ground truth and an unfair baseline, and its flicker measurement was unreliable, so its numbers could not say which method was better.
4. **Ground truth.** Building pairs of low- and high-resolution target tubes, by cropping targets from good-quality video and degrading them in a controlled way.
5. **Fair baselines.** Running every method on exactly the same low-resolution input.
6. **The video prior.** Replacing the image prior with a frozen Stable Video Diffusion model and designing how the low-resolution target steers it.
7. **Choosing the data.** Analysing every object in DAVIS 2017 and selecting test and validation targets from the statistics.
8. **Tuning, then testing.** Picking the one free parameter on the validation set, then running all methods once on the test set.
9. **The demo.** Putting everything into an interactive app with region metrics.

3. Related Work

Diffusion Models

Denoising diffusion models [1] are trained to remove a small amount of Gaussian noise from images. To generate images, the model is provided with pure noise and denoises the image step-by-step. Latent diffusion [2] uses these diffusion models within an autoencoder to compress the data. The latent space has a factor of eight fewer pixels per side, allowing for high resolution and video generation on normal GPU hardware. All diffusion models used in this project follow the basis of these two concepts.

Stable Video Diffusion

SVD [3] is an adaptation of the Stable Diffusion 2.1 image model where every spatial block has had temporal convolutions and attentions added and the model was trained incrementally using a large set of curated videos. This allows image to video generation where one image can be used to generate a short video. It was selected as our prior due to having pre-trained temporal convolutions, the weights are openly available (under Stability AI's licence) and can be run using the diffusers library [18].

Diffusion Priors for Super-Resolution

StableSR [4] adapts a frozen Stable Diffusion model for image super-resolution. One of the concepts in the paper was a colour correction that retained the low frequency information of the input image. This was used as the third stage in our method. Upscale-A-Video [5] and STAR [7] are both examples of diffusion priors for real world video super-resolution. Upscale-A-Video uses extra temporal layers along with flow guided propagation while STAR is a diffusion prior that uses a text to video model with five billion parameters. Both models were trained end-to-end on full frames but were very heavy and did not fit on our one T4 GPU. This led us to look into a no-training solution. SDEdit [25] showed that if you start the reverse diffusion process with a partially noisy version of the input image, then the input content and structure is retained. This concept has been implemented in our method using a strength parameter on the video tube.

Other Non-Diffusion Models

BasicVSR++ [8] is a recurrent video super-resolution model that uses optical flow to align adjacent frames. It achieves good PSNR results but produces blurry images. RealBasicVSR [9] is an adaptation of the basic model for real world degradations. Real-ESRGAN [10] is a GAN based image super-resolution model trained on synthetically generated real world degradations (blur, resize, noise and compression). It was used as a frame based baseline and its degradation model was used to generate our real world test inputs.

Tracking and Evaluation

SAM 2 [12] can segment an object in a video given only a single prompt. It can also be used as a tracker in the demo. LPIPS [15] is a perceptual loss that uses deep features to calculate the distance between two images. This was used for evaluation. tLPIPS [26] is a temporal version of this. CLIP-IQA [19] and MUSIQ [20] are no-reference quality metrics.

4. Methods and Approaches

4.1 Before the prep review: Experiment 1

Firstly we wanted to know if using a cropped diffusion prior is more effective. To test this we used a 1280×720 video (Pexels Video 6655841 consisting of 212 frames at 30fps), and focused on a small "BAD VIBES" sign in the video. We reduced the amount of frames in the video to make it fit onto the free Colab GPU. This was done by taking every 8th frame, resulting in 24 frames. We manually entered the box coordinates of the sign (600, 60, 180, 180), and used OpenCV's MIL tracker [24] thereafter.

This prior was the Stable Diffusion ×4 upscaler [21]. We performed this in half precision using CPU offloading. The targeted prior cropped the box with a 10 pixel buffer and performed the upscale using 30 steps of diffusion with a noise level of 60. We then pasted this into the base frame. The base frame performed the same upscale but not on the cropped area. We resized this to 128 pixels on the long side to fit into memory. This resulted in an output size of 288×512. The number of steps used were 20. To avoid flickering, we warped the previous output to the current frame using Farnebäck optical flow [27]. We then merged the two images together using a weight of 0.6. We used a small Gradio app to allow users to upload their video and enter the box.

This demonstrated that cropping does help the prior focus on the box. Upon looking at the results closer though, we cannot tell if our method is actually better due to:

- There was no ground truth, so we could only measure how different the two outputs were, not which one was closer to reality.
- The baseline was handicapped from the start, because it began from a 128-pixel version of the frame.
- With 8-frame gaps between frames, the optical flow was unreliable, and our own 0.6 blending artificially smoothed the output, so the flicker metric favoured our branch for the wrong reason.
- We also found that guidance had done nothing at all. With an empty prompt, the two branches of classifier-free guidance see identical inputs, so a guidance scale of 9 only doubled the computation.

These four problems shaped everything that followed.

4.2 After the prep review: the rebuilt pipeline

We rebuilt the project around three changes: real ground truth, baselines that receive exactly the same input, and a video prior in place of the image prior. Figure 1 shows the resulting pipeline. The user's box is tracked through the low-resolution clip, a fixed-size square tube is cut around the target, the tube is restored by the frozen video model, and the result is blended back into the upscaled frame.

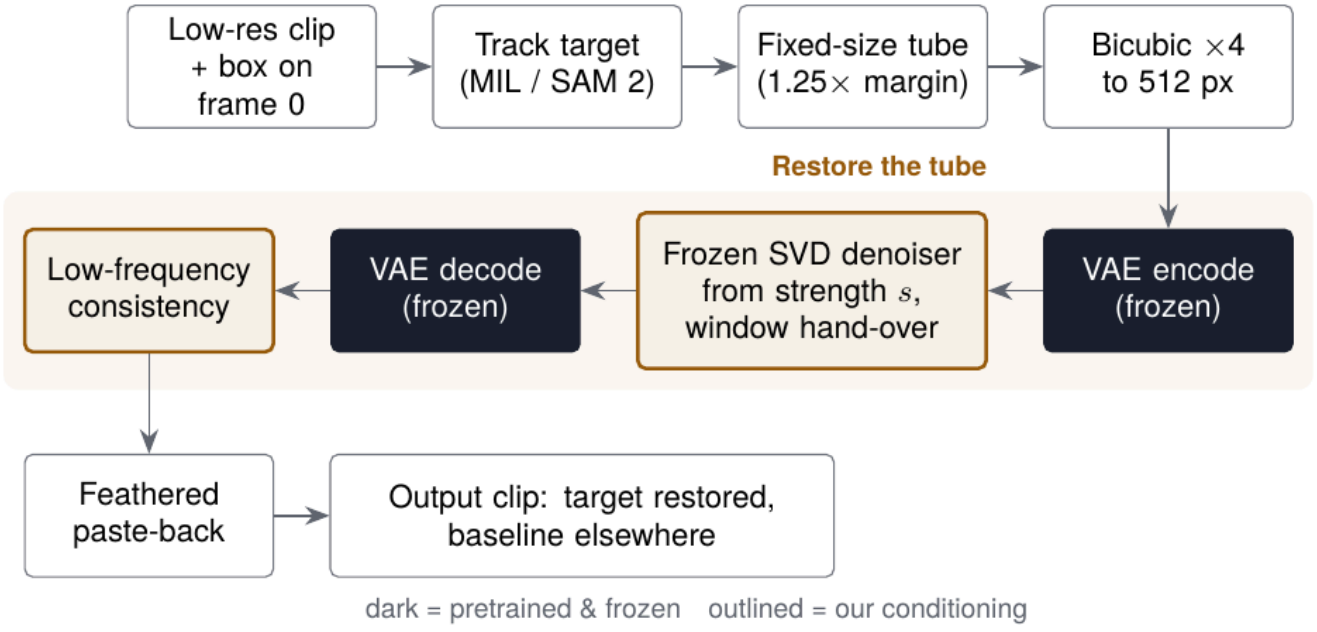


Figure 1. The targeted restoration pipeline. Dark boxes are pretrained and frozen; outlined boxes are the conditioning steps we designed. No component is trained.

Cutting a good tube

Care was taken while cutting out the tube to ensure the comparison was fair. The cropped image remains constant across a number of frames, only the middle follows the target movement, and is smoothed over 5 frames. If the scale changed between each frame it would give an unwanted zooming effect which would look like flickering. The side length of the window is 1.25 times the maximum extent of the target. This means that the model can see a little of surrounding area. It is then rounded up to be a multiple of 32 high resolution pixels. This means the low resolution tube can be a multiple of 8. The ground truth is never upscaled. Any targets larger than 512 pixels are reduced in size meaning they are still clear. Any targets smaller than 128 pixels are skipped since when reduced to 32 pixels there is too little information to reconstruct.

The frozen prior: Stable Video Diffusion

For our video generation task we use the image to video model stabilityai/stable-video-diffusion-img2vid [3] [28]. All three components are fixed, no gradient computed, and run in half precision. The images are compressed using an autoencoder, turning each image

into four channel latent representations of 1/8 the resolution. This latent is decoded using a cross-frame decoder. The UNet uses the spatial components from Stable Diffusion 2.1. After each block we apply time convolution and attention. Lastly we have a CLIP image encoder to describe the conditioning image.

In standard operation SVD will take an input image and condition each frame in the generated clip on this image. This conditioning is done both by inserting the CLIP embedding into the UNet via cross attention, and by adding the autoencoder latents as 4 additional channels to all the noisy latents. We also insert a few scalar values such as frame rate, motion intensity and noise level added to the conditioning image. The sampling is done using an Euler sampler, with v-prediction in the EDM paradigm. We use 25 sampling steps.

SVD was selected from our preparation review as it is open source, exhibits strong learned motion, is supported by diffusers, and can be run in 16GB GPU memory with our tube sizes. The other options were either designed for full frames rather than crops (Upscale-A-Video), too big to fit on a T4 (STAR backbone) or do not have as strong temporal model (MGLD-VSR [6] uses an image prior).

Steering the frozen model with the target

SVD was trained to *invent* a video from a single frame, not to restore one. We steer it with the low-resolution target in three ways, none of which requires training.

Starting from the input instead of from noise. We upsample the tube four times with bicubic interpolation, resize it to a working size of 512 pixels and encode it with the autoencoder into latents z_0. Rather than starting from pure noise, we add noise only up to an intermediate point of the schedule and run just the remaining denoising steps:

```
z_{t_s} = z_0 + \sigma_{t_s}\,\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \qquad t_s = \operatorname{round}\big(N(1-s)\big)
```

The strength s between 0 and 1 controls how much the model may change. A low strength keeps the output close to the input; a high strength gives the prior more freedom. With N = 25 steps and s = 0.3, sampling begins at step 18 and runs 7 denoising steps.

Passing a restored frame between windows. SVD handles 14 frames at a time, while our tubes are 27 frames long. Consecutive windows therefore overlap by one frame, and each new window is conditioned on the already restored last frame of the previous window, which also serves as its first starting latent. Details the model found in one window carry over into the next instead of being reinvented, which avoids a visible jump at the window boundary.

Keeping the input's coarse structure. After decoding, we take the low frequencies from the bicubic input and only the high frequencies from the model, using a Gaussian blur G with σ = 3 pixels:

```
\hat{y} = G_\sigma\!\left(y_{\text{bicubic}}\right) + \Big(y_{\text{SVD}} -
G_\sigma\!\left(y_{\text{SVD}}\right)\Big)
```

This follows StableSR's colour correction [4]. The model may add fine texture, but it cannot shift colours or move shapes away from what the low-resolution input shows, which is our main safeguard against invented content.

The restored tube is then resized to the original crop size and blended into the upscaled frame. The blending weight is 1 in the middle of the tube and falls smoothly to 0 over its outer 15%, so no edge is visible.

Losses and training

Nothing in this project is trained or fine-tuned, so there is no loss function, optimiser or learning-rate schedule to report. The frozen SVD model was trained by its authors with a denoising objective: given a noised latent and the noise level, the network predicts a "v" target, with EDM-style weighting across noise levels. We only use this learned denoiser at inference time. Three reasons led us to a training-free approach: we had a single T4 GPU, no paired dataset of target tubes exists at anything like the scale SVD was trained on, and the project description asks for a frozen model conditioned on the target region.

Baselines

Every baseline receives exactly the same low-resolution tube and produces an output four times larger, of the same size as ours.

- **Bicubic interpolation** with OpenCV: the lower bound, and also the starting point of our method.
- **Real-ESRGAN, per frame**: the RealESRGAN_x4plus model [10], [30], loaded with the spandrel library [29] in half precision and applied to each frame on its own.
- **The SD ×4 image prior**: the model from Experiment 1, applied frame by frame with 20 steps, noise level 60, an empty prompt, no guidance and the same random seed in every frame. Comparing it with our method isolates the effect of using a video prior instead of an image prior.

5. Data

5.1 Datasets

Dataset	What it contains	How we used it
DAVIS 2017 TrainVal, 480p [22]	90 videos (60 train, 30 validation) of JPEG frames at 854×480, with a mask for every object in every frame; about 794 MB	Test and validation targets
Pexels video 6655841 [32]	One clip, 1280×720, 30 fps, 212 frames	Experiment 1, and a text target in the demo
Own phone recordings	Short clips with no ground truth	The demo's real-world mode

Table 1. Datasets used. DAVIS was downloaded from the official site (davischallenge.org).

DAVIS was an obvious choice due to the precise mask on each object in every frame. As the target location is exactly known we can avoid any errors made by the tracker when comparing super-resolution methods. It is also a popular benchmark containing a variety of object sizes and speeds to choose from. The dataset is quick to download and our method does not require any training.

We also looked into REDS [13], which does not contain masks for objects and is a large download. CelebV-HQ [14] requires clips to be downloaded from YouTube. YouTube-VOS is large and requires registration. We were unable to get confirmation on license plate videos in time.

5.2 Target Selection

A script was written to read only the DAVIS annotation masks. For each object we calculated the size, proportion of frame they take up, speed of movement, number of frames they are seen in and whether they meet our tube requirements for a full 27 frame window.

For an object to be a true target, it must have a tube of less than or equal to 75% of the frame height. Otherwise it would just be full frame super-resolution. We selected one object per video from the valid objects in the DAVIS validation set. Eight objects were selected for testing with an even selection of slow to fast moving objects. Three more objects from different videos were selected for validation. This ensures that none of the test videos will be seen during tuning. As the objects were only selected based on annotations there is no bias towards any particular method.

The analysis shows how important this filtering is. Only 58 out of the 61 objects in the 30 validation videos met the basic tube requirements (2 were too small and 1 was never fully visible within the 27 frame window). Only 21 objects were true targets. DAVIS was designed for segmentation so most objects are large. The median target was 264 pixels wide in a 480 pixel height frame meaning they take up 11% of the image. The largest tubes cover the full height of the frame (see Figure 2, right). All objects in validation are visible in all their frames. The median object moved 5.6 pixels per frame and fastest 10% moved more than 12 pixels.

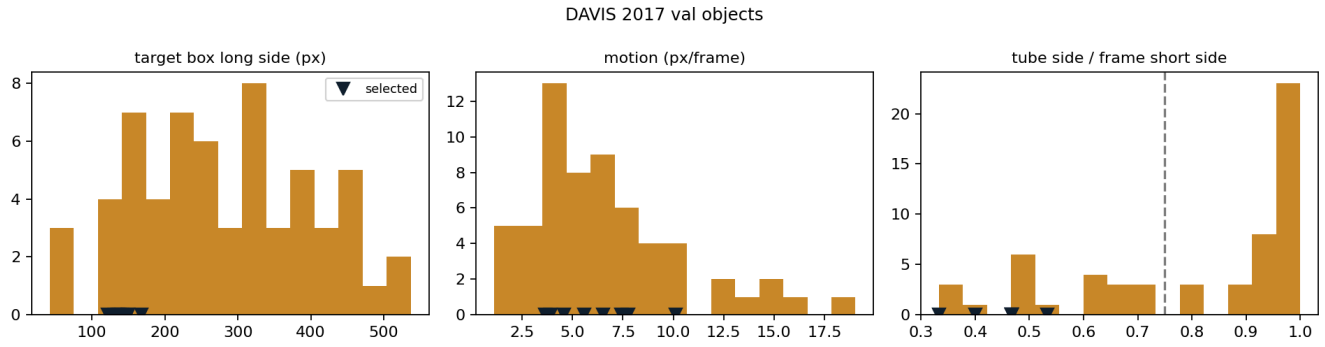


Figure 2. Statistics of the 61 objects in the DAVIS 2017 validation split: target size, motion and tube size relative to the frame. Triangles mark the eight selected test targets; the dashed line is the 75% limit for a genuinely targeted object.

Set	Video : object	Box side (px)	Tube side (px)	Tube / frame height	Motion (px/frame)
Test	gold-fish : 2	150	192	0.40	3.6
Test	paragliding-launch : 1	136	224	0.47	3.9
Test	mbike-trick : 2	137	160	0.33	4.6
Test	horsejump-high : 2	122	224	0.47	5.6
Test	drift-chicane : 1	141	224	0.47	6.5
Test	dogs-jump : 1	168	224	0.47	7.4
Test	bmx-trees : 1	132	256	0.53	7.8
Test	kite-surf : 3	150	224	0.47	10.1
Val	pigs : 1	248	320	0.67	3.8
Val	scooter-black : 1	185	288	0.60	7.0
Val	parkour : 1	230	288	0.60	9.5

Table 2. The selected targets. Box side is the median over frames; the tube includes the 1.25× margin and rounding to 32 px.

The eight test targets are all well inside the frame, with tubes between a third and a half of the frame height (160 to 256 pixels, so 40 to 64 pixels at low resolution), and they cover a wide range of motion, from a slowly drifting goldfish to a kite surfer moving ten pixels

per frame.

5.3 Making low- and high-resolution pairs

For each selected target, we cut a 27-frame high-resolution tube from the 480p frames using the rules above, and created its low-resolution input by degrading it four times. We cut 16 extra pixels of context around the tube, degraded them together with it, and then threw them away, so that blur and compression artefacts at the border do not fall on the target. Every tube was degraded in two ways:

- **Bicubic:** smooth, antialiased bicubic downsampling, the standard "BI" setting on which BasicVSR++ was trained.
- **Real-world:** the kind of damage Real-ESRGAN [10] simulates. Gaussian blur (σ between 0.2 and 2.5, stretched in one direction half the time), random resizing, Gaussian or Poisson noise and JPEG compression (quality 40–95), followed with probability 0.7 by a second round of the same, with the order of the operations shuffled, and finally H.264 video compression (CRF 18–30) with probability 0.5. We used milder ranges than the Real-ESRGAN paper, because our low-resolution targets are only 32 to 128 pixels across and the original settings can wipe them out.

One detail matters a lot for video: we draw the degradation settings **once per clip** and reuse them for every frame. Only the random noise changes from frame to frame. If the amount of blur or compression changed from frame to frame, the input itself would flicker, and methods that try to be stable over time would be unfairly penalised.

In total, the test set has 16 tubes (8 targets × 2 degradations) and the validation set 6 tubes (3 targets × 2 degradations), each 27 consecutive frames long. Test and validation targets come from different videos.

6. Experiments

6.1 Protocol

We tuned on the validation set and reported on the test set, running the test only once. The only tuned parameter was the strength s; everything else was fixed beforehand. All methods saw the same low-resolution tubes, produced outputs of the same size, and were scored against the same high-resolution ground truth.

6.2 Settings

Setting	Value
Scale factor	×4

Setting	Value
Tube length	27 consecutive frames, i.e. two SVD windows (14 frames + 13 new ones)
Diffusion steps	25 (Euler sampler, v-prediction)
Strength (chosen on validation)	0.3, i.e. 7 denoising steps
Working resolution	512 px, then resized to the tube's ground-truth size
Conditioning noise / motion / fps	0.02 / 127 / 7
Low-frequency blur σ	3 px
Random seed	0 for every window
SD ×4 baseline	20 steps, noise level 60, empty prompt, guidance 1, same seed per frame
Real-ESRGAN	RealESRGAN_x4plus, half precision, per frame
Optimiser, learning rate, schedule	Not applicable (nothing is trained)

Table 3. Settings used in all experiments.

6.3 Hardware and software

All experiments ran in Kaggle notebooks with two NVIDIA Tesla T4 GPUs (16 GB each); the test tubes were split between the two GPUs and processed in parallel. The code is written in Python with PyTorch and diffusers, plus spandrel, OpenCV, scikit-image, lpips and pyiqa. Table 4 lists the measured cost of each method on one T4.

Method	Time per frame	Time per 27-frame tube	Peak GPU memory
Real-ESRGAN	0.05 s	1.4 s	0.2 GB
SD ×4 image prior	1.24 s	33 s	2.4 GB
Ours, strength 0.5 (timing run)	18.4 s	497 s	7.0 GB
Ours, strength 0.3 (final test)	about 10 s	258–274 s	under 7 GB

Table 4. Runtime and memory on one NVIDIA T4. The final-test figures are averages over the eight tubes processed on each of the two GPUs.

Our method is by far the slowest. Even at strength 0.3 it needs about four and a half minutes per tube, roughly 200 times longer than Real-ESRGAN, because every window runs a 1.5-billion-parameter video UNet over 14 frames at 512 pixels. It does, however, fit comfortably in a T4's 16 GB.

The code, the executed Kaggle notebook behind every number in this report, and the figures are available at github.com/avishiii/Targeted-Video-SR.

6.4 How we measure

Every metric is computed only on the target:

- **PSNR and SSIM** measure fidelity to the real scene. We compute them on the brightness (Y) channel, inside the ground-truth object mask.
- **LPIPS** measures perceptual distance to the ground truth using deep network features, averaged inside the mask; lower is better.
- **tLPIPS** measures flicker. For each pair of consecutive frames, it compares how much the output changes with how much the ground truth changes, so real motion is not counted as flicker; lower is better.
- **Warping error** also measures temporal stability. We move the previous output frame onto the current one using optical flow computed on the *ground truth*, so that every method is judged against the same motion, and measure what difference remains. We also report the ground truth's own warping error as a reference: a value far below it does not mean "very stable", it means "smoother than reality".
- **CLIP-IQA and MUSIQ** are no-reference quality scores; we report them for the ground truth too, for comparison.
- **Seam score** (demo only) compares image gradients in a thin band along the tube's border with the ground truth, to check that the blending leaves no visible edge.

7. Results

7.1 Choosing the strength on the validation set

We ran our method at strengths 0.3, 0.5 and 0.7 on the six validation tubes (Table 5).

Degradation	Strength	PSNR ↑	SSIM ↑	LPIPS ↓	tLPIPS ↓	Warp
Bicubic	0.3	**26.42**	**0.727**	**0.499**	**0.019**	12.29
Bicubic	0.5	23.08	0.580	0.546	0.033	12.90
Bicubic	0.7	22.53	0.544	0.563	0.054	11.33
Real-world	0.3	**22.63**	**0.544**	0.723	**0.047**	5.27

Degradation	Strength	PSNR ↑	SSIM ↑	LPIPS ↓	tLPIPS ↓	Warp
Real-world	0.5	21.44	0.493	0.688	0.055	5.44
Real-world	0.7	21.09	0.471	**0.657**	0.078	6.96

Table 5. Validation results for different strengths (3 DAVIS targets per degradation, inside the target mask; warping error ×10⁻³).

The bicubic degradation follows an intuitive pattern where the lowest strength performs best across all metrics. However, by increasing the flexibility of the model we see that the texture of the real degradation looks more realistic (LPIPS score drops from 0.723 to 0.657). The output however is less faithful and contains greater flicker. This is a classic example of the perceptual quality vs fidelity dilemma.

We had decided before testing that we would choose the strength with the lowest average LPIPS. This turned out to be 0.7 however it only differed from 0.3 by 0.001 (from 0.610 to 0.611). This is not significant in terms of 6 tubes. To overcome this we chose 0.3 due to it having an increased average PSNR (2.7dB), less flicker (tLPIPS score 0.033 compared to 0.066) and requiring fewer denoising steps (7 compared to 17). All results were run using s = 0.3.

7.2 Comparison with the baselines

Figure 3 summarises the two most important measurements, and Tables 6 and 7 give the full results on the 16 test tubes.

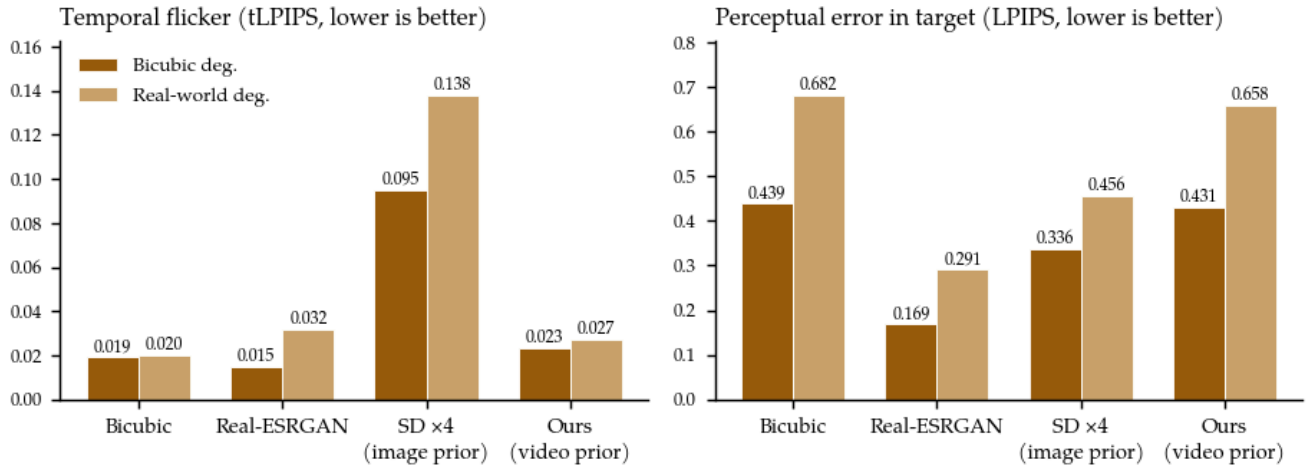


Figure 3. Flicker (left) and perceptual error inside the target (right) on the test set, for both degradations. The video prior keeps flicker close to bicubic interpolation, far below the per-frame image prior, but adds little perceptual detail.

Method	PSNR ↑	SSIM ↑	LPIPS ↓	tLPIPS ↓	Warp	CLIP-IQA ↑	MUSIQ ↑
Bicubic	**24.99**	**0.704**	0.439	0.019	3.12	0.446	18.78

Method	PSNR ↑	SSIM ↑	LPIPS ↓	tLPIPS ↓	Warp	CLIP-IQA ↑	MUSIQ ↑
Real-ESRGAN (per frame)	23.07	0.685	**0.169**	**0.015**	9.25	**0.538**	**46.10**
SD ×4 image prior (Exp. 1)	21.82	0.577	0.336	0.095	13.38	0.444	42.75
Ours: SVD video prior	24.56	0.682	0.431	0.023	2.75	0.322	19.19
Ground truth	–	–	–	–	*6.00*	*0.512*	*43.97*

Table 6. Test results, bicubic degradation (8 tubes). PSNR, SSIM and LPIPS inside the target mask; the other metrics on the target tube; warping error ×10³. The ground-truth row is for reference.

Method	PSNR ↑	SSIM ↑	LPIPS ↓	tLPIPS ↓	Warp	CLIP-IQA ↑	MUSIQ ↑
Bicubic	**22.15**	0.549	0.682	**0.020**	2.61	0.324	16.25
Real-ESRGAN (per frame)	21.38	**0.558**	**0.291**	0.032	9.30	**0.529**	**40.69**
SD ×4 image prior (Exp. 1)	20.35	0.457	0.456	0.138	12.41	0.393	38.17
Ours: SVD video prior	22.02	0.540	0.658	0.027	2.26	0.213	16.07
Ground truth	–	–	–	–	*6.00*	*0.512*	*43.97*

Table 7. Test results, real-world degradation (8 tubes).

7.3 What the results show

The flicker issue has been solved in the video prior. This answers the main research question of the project and clearly shows how much less flickering there is in our method compared to the image prior. We get approximately 4-5 times less flicker (tLPIPS values go from 0.095 to 0.023 with bicubic degradation and 0.138 to 0.027 with real world degradation). The results are also much closer to the ground truth with an increase of 2.7 dB and 1.7 dB in PSNR. The image prior has a warping error of between 12-13, which is roughly double that of the ground truth (6.0). Our method is much less than this.

However, it does not add much additional information beyond what the bicubic method provides. It has only a small improvement from the initial input (LPIPS of 0.439 goes down to 0.431, and 0.682 goes down to 0.658). The PSNR value decreases by 0.1-0.4 dB. The CLIP-IQA score is the worst of all methods because the targets still appear blurry. At a strength of 0.3 the model doesn't really make any changes to the tube. There are a few reasons for this. Only 7 out of 25 denoising steps were carried out, and at low noise levels so the model didn't really change much. The low frequency step intentionally caused anything coarser than around 3 pixels to be fixed to what the bicubic input was. Also, the SVD was trained to create motion from a high quality frame so it wasn't trained to deblur, therefore when presented with a smooth realistic video at low noise there isn't much incentive to make it sharper.

Real-ESRGAN gives the sharpest images. The GAN on a per-frame basis is clearly the best on perceptual quality (LPIPS was 0.169 compared to 0.431 for our method with bicubic degradation). Its MUSIQ score (46.1) is even higher than the ground truth's (44.0), which suggests it is over-sharpening. The warping error (9.3) is worse than the ground truth meaning it has added some temporal noise. The video also has more flicker than our method (tLPIPS of 0.032 vs 0.027) with real world degradation.

Our warping error and bicubic's are both lower than the ground truth's 6.0 (2.3-2.8 for ours, 2.6-3.1 for bicubic). Less than 6.0 is not a good thing in this case. This means the outputs don't change between frames as much as a real video would because there is no detailed information in the videos. In this case, the tLPIPS which compares the amount the output changes between frames compared to the ground truth is a better measurement of flicker.

Overall the issues with flicker in the image prior have been solved with the frozen video prior. However, it does not provide enough extra information to match a strong per-frame GAN due to the conservative conditioning. A stronger conditioning which takes into account the target would be an obvious choice to improve this. This is where our plan for novelty comes in.

7.4 Qualitative results

Figure 4 shows the middle frame of three test tubes for every method.

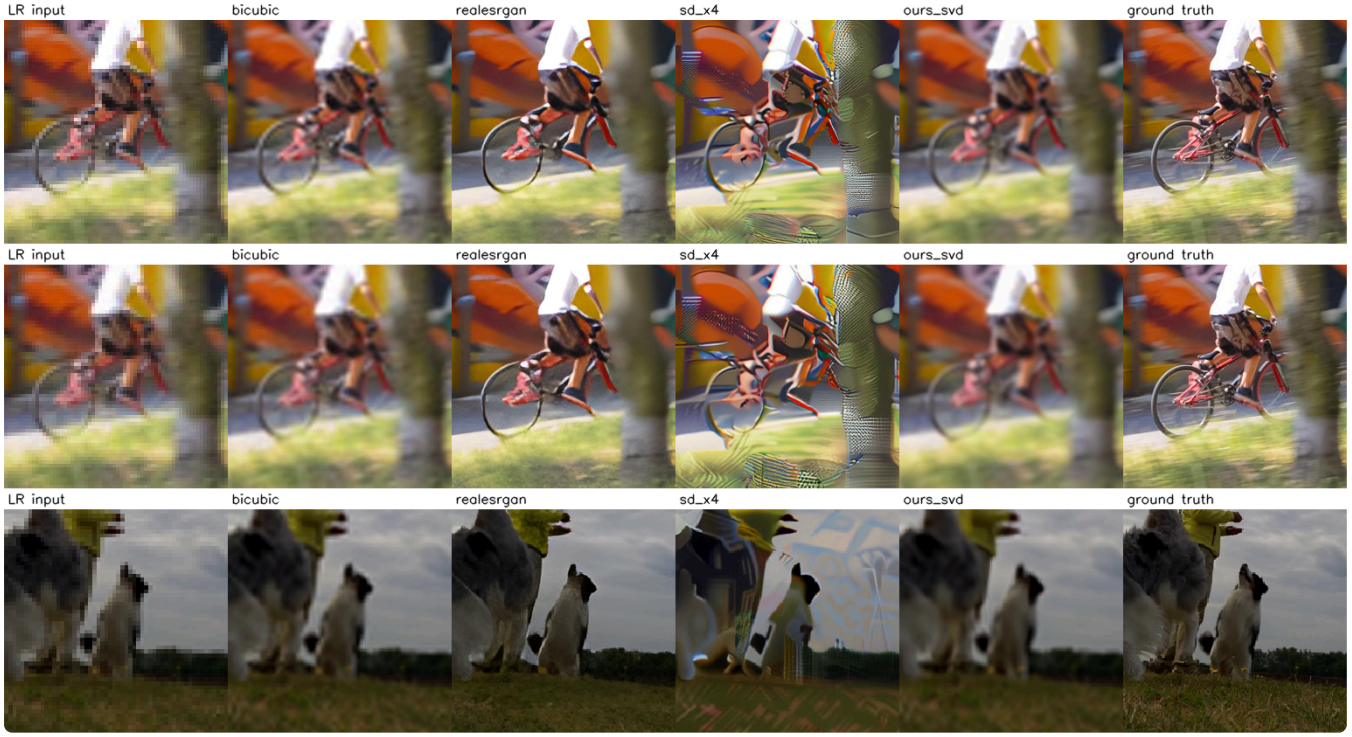


Figure 4. Middle frame of three test tubes. Top: bmx-trees, bicubic degradation. Middle: bmx-trees, real-world degradation. Bottom: dogs-jump, bicubic degradation. Columns: low-resolution input, bicubic, Real-ESRGAN, SD ×4 image prior, ours, ground truth.

The pictures confirm the numbers and make the trade-offs concrete. Bicubic interpolation and our method look almost the same: shapes and colours are right, but the bicycle's spokes and the dog's fur stay blurred. Real-ESRGAN gives the crispest edges, recovering the wheel and the rider's shoes, but it turns fine texture such as the grass into smooth, painted-looking patches. The SD ×4 image prior shows the danger of an unconstrained generative model most clearly. It covers the tree trunk with an invented woven pattern, distorts the bicycle, and in the dogs-jump frame replaces the person and the sky behind the dog with structures that do not exist in the scene. This is exactly the kind of hallucination our low-frequency constraint prevents, at the price of the softness visible in our column.

8. Interface

The demo is a web application using Gradio [23] (app.py). This runs on a GPU notebook and can be accessed via a public link, therefore making it accessible anywhere through a browser.

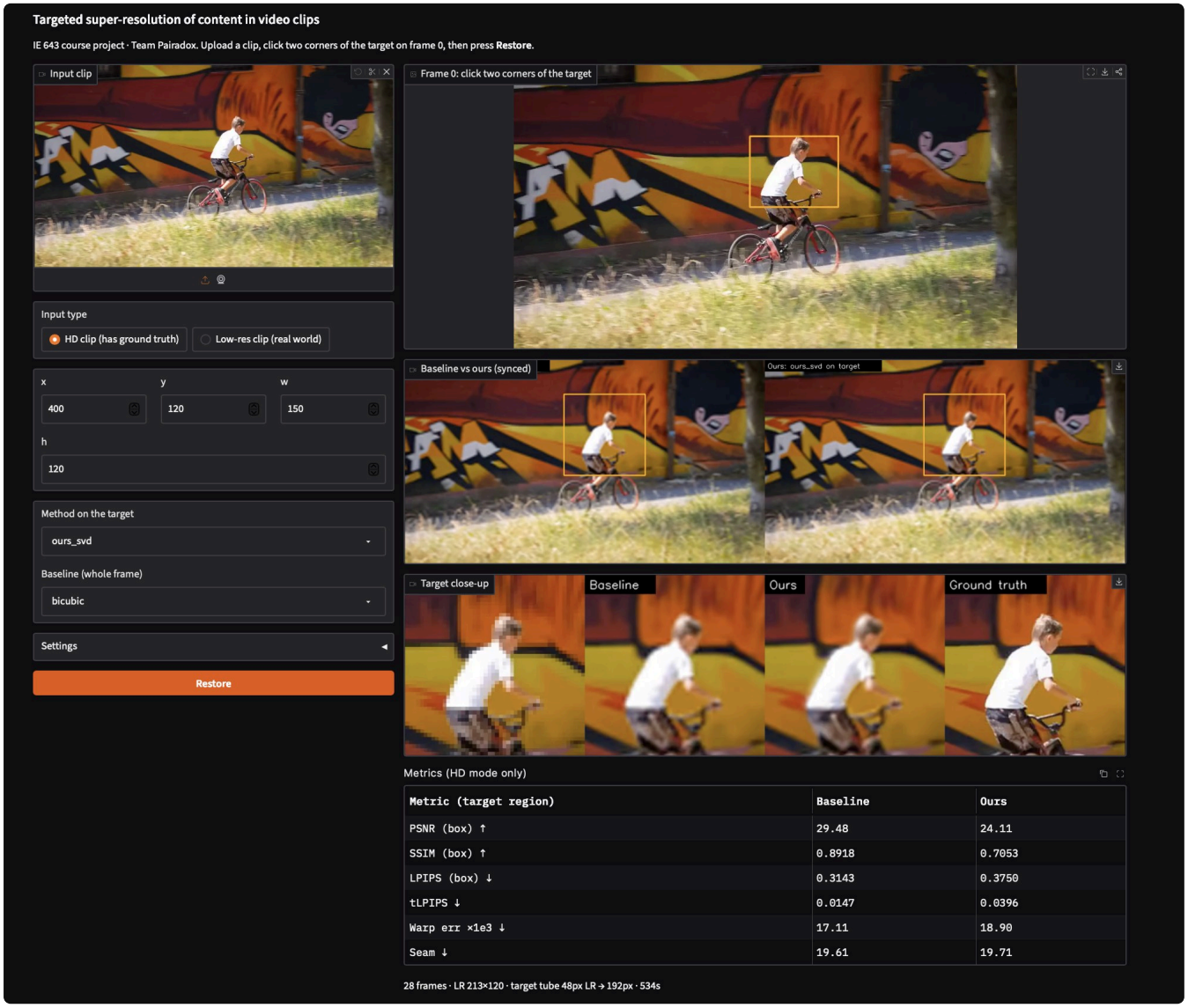
The user uploads a video. The first frame is displayed. The user clicks on two opposite corners to select an area of interest. They can then adjust how the area of interest is defined. Once defined the user selects which method to use (by default our SVD method), the baseline (bicubic/Real-ESRGAN) to apply to the entire frame, which tracker to use, how many frames to process and how much the model may change the target (strength). Once these are selected the user can click restore.

The application has two modes. HD clip mode uses the uploaded video as the ground truth. We degrade the video, restore it and calculate a score based on performance against the ground truth. Low resolution clip mode uses the uploaded video as the input. Any inputs greater than 480 pixels are scaled down before processing to allow the outputs to be a manageable size (four times larger). There is no ground truth.

In the background, the application will track the area of interest using either OpenCV MIL or optionally SAM 2. This is done on the low resolution frames (as a real user would do). The video will then be cropped according to the same criteria as our experiments were. This will be restored and the entire frame will be upscaled using the selected baseline. Finally the restored video will be overlaid. As both videos will have the same background, we can be confident that we are only seeing differences within the area of interest due to targeted restoration.

The application will output a side by side video of the baseline and our result, including both in one file to ensure they are aligned. This will include an outline of the area of interest. The area of interest will also be zoomed into, showing the low resolution input, baseline, our restored video and ground truth (if using HD mode). If using HD mode a table will show the metrics for just the area of interest (PSNR, SSIM, LPIPS in the box).

These will include the metrics for both the baseline and our method (tLPIPS, warping error and seam score). The videos will be encoded in H.264 so can be played directly in the browser. An example of the application after running can be seen in Figure 5.



Metric (target region)	Baseline	Ours
PSNR (box) ↑	29.48	24.11
SSIM (box) ↑	0.8918	0.7053
LPIPS (box) ↓	0.3143	0.3750
tLPIPS ↓	0.0147	0.0396
Warp err ×1e3 ↓	17.11	18.90
Seam ↓	19.61	19.71

Figure 5. The demo app on a 28-frame HD clip (bmx-trees) in HD mode with bicubic degradation. Left: the input, the target box (x, y, w, h) and the method choices. Right: frame 0 with the selected target, the synchronised baseline-versus-ours video, the target close-up (low-resolution input, baseline, ours, ground truth) and the target-region metrics. This run used a strength of 0.5.

The metrics in this example also illustrate the validation finding of Section 7.1. With a mild bicubic degradation and a strength of 0.5, our method falls below plain bicubic interpolation on every metric (PSNR 24.1 against 29.5 dB, LPIPS 0.375 against 0.314), because the extra denoising steps alter content that bicubic interpolation already recovers well. This is why the reported experiments use s = 0.3, and why choosing the strength automatically from the degradation level is part of our novelty plan.

9. Plan for Novelty Assessment

The results highlight the area to improve. Our method performs well at keeping the target faithful and stable, however, it lacks extra detail. Our improvements will therefore focus on improving the perceptual quality in the target.

1. **Increased perceptual quality in the target (main aim).** This was the most obvious area we noticed to improve in our results. Our model performs very similarly to bicubic on LPIPS, CLIP-IQA and MUSIQ, and well below Real-ESRGAN. We will aim to increase the high frequency detail generated by the prior whilst maintaining fidelity and temporal consistency. We will change the static Gaussian used to constrain the low frequency part to a frequency sensitive fusion with an adaptive cutoff based on the degradation estimated. This will help avoid mid range features being pulled back too far towards the blurry input. Additionally, we will increase the effective strength used for denoising the high frequency part. This will help inject noise into the latent space detail part, allowing our model to generate new texture. Lastly, we will try to combine our model with Real-ESRGAN. This will provide us with a GAN restored image per frame. This will be used as our initialisation and conditioning frame for the SVD. Our model will have more detail thanks to the GAN and the video prior will no longer be generating detail but ensuring it is consistent across time. If we are successful, we should see a lower LPIPS score in our target and higher scores in MUSIQ and CLIP-IQA whilst maintaining the same or improved PSNR/tLPIPS.
2. **Adaptive strength.** The validation experiment highlighted that it is important to adjust the strength depending on the degradation. Lower strengths were needed for images with slight blur and higher strengths for heavily degraded real-world images. We will estimate the degradation of each tube using metrics such as blur and noise. Then we will automatically choose what strength was needed. We will then compare this with a static strength.
3. **Higher strength inside object.** Increasing the strength inside the object mask and decreasing it in the margin around the object will allow more detail to be added to the object whilst maintaining fidelity around the edges. This will also help with blending.
4. **At every step.** Rather than enforcing consistency once after decoding, we will encourage our model to generate a clean estimate by nudging our estimates closer to the low resolution target at every step of denoising.
5. **Small adaptation layer.** Adding a light encoder to pass our low res tube through to the frozen UNet. This would be trained on tubes from our DAVIS training split. This will show us how much benefit we gain from some training. It will also allow us to have a trained model within our project.
6. **Accuracy.** We can check how accurate our content is by looking at OCR accuracy on text targets and identity similarity on faces using ArcFace [17].

10. Conclusion

The goal in this project was to upscale a user selected area of the video by a factor of 4. This is done by tracking the target region using our pipeline, upsampling the fixed size tube using a frozen Stable Video Diffusion model. This is guided by the low-res input and seamlessly blended in. No training is performed.

We conducted a fair evaluation by creating ground truth tubes from the DAVIS 2017 dataset. We picked test tubes using only information from the dataset statistics. A single parameter was optimised using a validation split. All methods were evaluated purely on how well they reconstructed the target. The video prior improved upon the frame-wise image prior by 4-5x in terms of reducing flickering, and also kept much closer to the original. However, due to us being conservative with our conditioning, there was little extra detail in comparison to bicubic interpolation. The frame-wise GAN still outperformed in terms of perceptual quality. The validation study showed how there is a clear trade-off between perception and fidelity. By providing more freedom to the model we get more realistic textures, but less fidelity and consistency.

There are some limitations to the project. The test set was quite small (16 tubes). The degradation was synthetic, the ground truth was 480p, the sampling was slow on the T4 GPU. There was also no fine-tuning. The interactive demo allows users to select their own target region within their own video, and view how our method performs against the baseline. In the novelty phase we aim to make the target region clearly sharper. This requires the region to be consistent and remain similar to the input.

References

[1] Jonathan Ho, Ajay Jain and Pieter Abbeel. Denoising Diffusion Probabilistic Models, Advances in Neural Information Processing Systems (NeurIPS), 2020. https://arxiv.org/abs/2006.11239

[2] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser and Björn Ommer. High-Resolution Image Synthesis with Latent Diffusion Models, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022. https://arxiv.org/abs/2112.10752

[3] Andreas Blattmann, Tim Dockhorn, Sumith Kulal, Daniel Mendelevitch, Maciej Kilian, Dominik Lorenz, Yam Levi, Zion English, Vikram Voleti, Adam Letts, Varun Jampani and Robin Rombach. Stable Video Diffusion: Scaling Latent Video Diffusion Models to Large Datasets, arXiv preprint, 2023. https://arxiv.org/abs/2311.15127

[4] Jianyi Wang, Zongsheng Yue, Shangchen Zhou, Kelvin C.K. Chan and Chen Change Loy. Exploiting Diffusion Prior for Real-World Image Super-Resolution, International Journal of Computer Vision (IJCV), 2024. https://arxiv.org/abs/2305.07015

[5] Shangchen Zhou, Peiqing Yang, Jianyi Wang, Yihang Luo and Chen Change Loy. Upscale-A-Video: Temporal-Consistent Diffusion Model for Real-World Video Super-Resolution, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024. https://arxiv.org/abs/2312.06640

[6] Xi Yang, Chenhang He, Jianqi Ma and Lei Zhang. Motion-Guided Latent Diffusion for Temporally Consistent Real-world Video Super-resolution, European Conference on Computer Vision (ECCV), 2024. https://arxiv.org/abs/2312.00853

[7] Rui Xie, Yinhong Liu, Penghao Zhou, Chen Zhao, Jun Zhou, Kai Zhang, Zhenyu Zhang, Jian Yang, Zhenheng Yang and Ying Tai. STAR: Spatial-Temporal Augmentation with Text-to-Video Models for Real-World Video Super-Resolution, IEEE/CVF International Conference on Computer Vision (ICCV), 2025. https://arxiv.org/abs/2501.02976

[8] Kelvin C.K. Chan, Shangchen Zhou, Xiangyu Xu and Chen Change Loy. BasicVSR++: Improving Video Super-Resolution with Enhanced Propagation and Alignment, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022. https://arxiv.org/abs/2104.13371

[9] Kelvin C.K. Chan, Shangchen Zhou, Xiangyu Xu and Chen Change Loy. Investigating Tradeoffs in Real-World Video Super-Resolution, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022. https://arxiv.org/abs/2111.12704

[10] Xintao Wang, Liangbin Xie, Chao Dong and Ying Shan. Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data, IEEE/CVF International Conference on Computer Vision Workshops (ICCVW), 2021. https://arxiv.org/abs/2107.10833

[11] Lvmin Zhang, Anyi Rao and Maneesh Agrawala. Adding Conditional Control to Text-to-Image Diffusion Models, IEEE/CVF International Conference on Computer Vision (ICCV), 2023. https://arxiv.org/abs/2302.05543

[12] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár and Christoph Feichtenhofer. SAM 2: Segment Anything in Images and Videos, International Conference on Learning Representations (ICLR), 2025. https://arxiv.org/abs/2408.00714

[13] Seungjun Nah, Sungyong Baik, Seokil Hong, Gyeongsik Moon, Sanghyun Son, Radu Timofte and Kyoung Mu Lee. NTIRE 2019 Challenge on Video Deblurring and Super-Resolution: Dataset and Study, IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2019. https://seungjunnah.github.io/Datasets/reds.html

[14] Hao Zhu, Wayne Wu, Wentao Zhu, Liming Jiang, Siwei Tang, Li Zhang, Ziwei Liu and Chen Change Loy. CelebV-HQ: A Large-Scale Video Facial Attributes Dataset, European Conference on Computer Vision (ECCV), 2022. https://arxiv.org/abs/2207.12393

[15] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman and Oliver Wang. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018. https://arxiv.org/abs/1801.03924

[16] Haoning Wu, Erli Zhang, Liang Liao, Chaofeng Chen, Jingwen Hou, Annan Wang, Wenxiu Sun, Qiong Yan and Weisi Lin. Exploring Video Quality Assessment on User Generated Contents from Aesthetic and Technical Perspectives, IEEE/CVF International Conference on Computer Vision (ICCV), 2023. https://arxiv.org/abs/2211.04894

[17] Jiankang Deng, Jia Guo, Niannan Xue and Stefanos Zafeiriou. ArcFace: Additive Angular Margin Loss for Deep Face Recognition, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019. https://arxiv.org/abs/1801.07698

[18] Hugging Face. https://huggingface.co/docs/diffusers, Accessed on: 25th October, 2026.

[19] Jianyi Wang, Kelvin C.K. Chan and Chen Change Loy. Exploring CLIP for Assessing the Look and Feel of Images, AAAI Conference on Artificial Intelligence, 2023. https://arxiv.org/abs/2207.12396

[20] Junjie Ke, Qifei Wang, Yilin Wang, Peyman Milanfar and Feng Yang. MUSIQ: Multi-scale Image Quality Transformer, IEEE/CVF International Conference on Computer Vision (ICCV), 2021. https://arxiv.org/abs/2108.05997

[21] Robin Rombach and Patrick Esser. https://huggingface.co/stabilityai/stable-diffusion-x4-upscaler, Accessed on: 25th October, 2026.

[22] Jordi Pont-Tuset, Federico Perazzi, Sergi Caelles, Pablo Arbeláez, Alexander Sorkine-Hornung and Luc Van Gool. The 2017 DAVIS Challenge on Video Object Segmentation, arXiv preprint, 2017. https://arxiv.org/abs/1704.00675

[23] Abubakar Abid, Ali Abdalla, Ali Abid, Dawood Khan, Abdulrahman Alfozan and James Zou. Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild, arXiv preprint, 2019. https://arxiv.org/abs/1906.02569

[24] OpenCV team. https://docs.opencv.org/4.x/, Accessed on: 25th October, 2026.

[25] Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu and Stefano Ermon. SDEdit: Guided Image Synthesis and Editing with Stochastic Differential Equations, International Conference on Learning Representations (ICLR), 2022. https://arxiv.org/abs/2108.01073

[26] Mengyu Chu, You Xie, Jonas Mayer, Laura Leal-Taixé and Nils Thuerey. Learning Temporal Coherence via Self-Supervision for GAN-based Video Generation, ACM Transactions on Graphics, 2020. https://arxiv.org/abs/1811.09393

[27] Gunnar Farnebäck. Two-Frame Motion Estimation Based on Polynomial Expansion, Scandinavian Conference on Image Analysis (SCIA), 2003.

[28] Stability AI. https://huggingface.co/stabilityai/stable-video-diffusion-img2vid, Accessed on: 25th October, 2026.

[29] chaiNNer-org. https://github.com/chaiNNer-org/spandrel, Accessed on: 25th October, 2026.

[30] Xintao Wang. https://github.com/xinntao/Real-ESRGAN, Accessed on: 25th October, 2026.

[31] Kaggle. https://www.kaggle.com/docs/notebooks, Accessed on: 25th October, 2026.

[32] Pexels. https://www.pexels.com/video/6655841/, free stock video under the Pexels License, Accessed on: 26th October, 2026.

[33] Avishi. https://github.com/avishiii/Targeted-Video-SR, Accessed on: 26th October, 2026.